# Solution: Nested List Weight Sum

## Problem Statement

You are given a `nestedList`. Each element of the `nestedList` contains either integers or other such nested lists.

The depth level is determined by how many levels of nested lists the integer is within. For example, each element of the list `[1, 1, [2], [2, [3]]]` represents the depth of itself.

Return the `sum of all integers` of the list after `multiplying` each integer with its `depth level`.

### 2. Example Generation

- **Example 1**:
  - Input: `[[2, [3, 4]], [5, 6], 7]`
  - Expected Output: `54`
  - Justification: The sum is calculated as 4 (2 at depth 2) + 9 (3 at depth 3) + 12 (4 at depth 3) + 10 (5 at depth 2) + 12 (6 at depth 2) + 7 (7 at depth 1).
- **Example 2**:
  - Input: `[1, [4, [6]]]`
  - Expected Output: `27`
  - Justification: The sum is 1 (depth 1) + 8 (4 at depth 2) + 18 (6 at depth 3).
- **Example 3**:
  - Input: `[5, [2, [3, [4]]], 1]`
  - Expected Output: `50`
  - Justification: The calculation is 5 (depth 1) + 4 (2 at depth 2) + 9 (3 at depth 3) + 16 (4 at depth 4) + 1 (depth 1).

## Solution

To solve this problem, a recursive approach is ideal. This approach works by traversing each element in the list. If the element is an integer, we multiply it by its depth and add it to the total sum. If the element is another list, we recursively process this list, increasing the depth by one.

This method is effective because it naturally handles the nested structure of the input and ensures that each integer is multiplied by its correct depth level. The recursion elegantly simplifies the process of diving into nested lists and accumulating the weighted sum.

### Step-by-Step Algorithm

- Define a method `nestedSum` which takes a nested list and an integer `depth`.
- Initialize a variable `totalSum` to 0.
- Iterate through each element in the nested list.
  - If the element is an integer, add it to `totalSum` multiplied by `depth`.
  - If the element is a list, recursively call `nestedSum` with the list and `depth + 1`, and add the result to `totalSum`.
- Return `totalSum`.

### Algorithm Walkthrough

Let's consider the input: `[[2, [3, 4]], [5, 6], 7]`.

1. Start with the outer list: `[[2, [3, 4]], [5, 6], 7]` with `depth = 1`.
2. The first element is `[2, [3, 4]]`. It's a nested list, so we process its elements:
   - `2` is an integer at `depth = 2`. Add `2 * 2 = 4` to `totalSum`.
   - `[3, 4]` is another nested list. Process its elements at `depth = 3`:
     - `3` is an integer. Add `3 * 3 = 9` to `totalSum`.
     - `4` is an integer. Add `4 * 3 = 12` to `totalSum`.
3. The second element is `[5, 6]`, another list at `depth = 2`. Process its elements:
   - `5` is an integer. Add `5 * 2 = 10` to `totalSum`.
   - `6` is an integer. Add `6 * 2 = 12` to `totalSum`.
4. The third element is `7`, a single integer at `depth = 1`. Add `7 * 1 = 7` to `totalSum`.
5. Summing all these, the total sum is `4 + 9 + 12 + 10 + 12 + 7 = 54`.

## Code

Java

```java
import java.util.ArrayList;
import java.util.List;

// NestedInteger class represents either a single integer or a nested list of integers
class NestedInteger {
    private Integer value;
    private List<NestedInteger> list;
    private boolean isSingleInteger;

    // Constructor for a single integer
    public NestedInteger(int value) {
        this.value = value;
        this.isSingleInteger = true;
    }

    // Constructor for an empty nested list
    public NestedInteger() {
        this.list = new ArrayList<>();
        this.isSingleInteger = false;
    }

```

Run Code Result Stdout

You must run your code first

Execute

## Complexity Analysis

- **Time Complexity**: `O(N)`, where N is the total number of elements in the list, including nested lists. This is because each element is processed exactly once.
- **Space Complexity**: `O(D)`, where D is the maximum depth of the nested lists. This is due to the recursive stack.

← Previous
Nested List Weight Sum (Medium)

Next →
Dot Product Of Two Sparse Vectors (Medium)

☐ Mark as Completed